11. Seguridad

PDF Reader v0.3.3 | Análisis 2026-09-29 | Referencia del release v0.3.3

Contenido

- Modelo de amenazas
- Controles comprobados
- Datos personales y privacidad
- Validación y sanitización
- Riesgos
- CORS, CSRF e inyección
- Actualizaciones de dependencias por CVE
- Secretos y respuesta
- Recomendaciones prioritarias

Modelo de amenazas

Los PDF, nombres y rutas son entradas no confiables. Los activos principales son el contenido privado del documento, el almacenamiento local, la integridad del original, la clave de firma Android y el entorno del usuario. No hay identidad, autenticación, roles ni sesiones porque no existe servicio multiusuario.

Controles comprobados

Superficie	Control
Alcance	no hay funciones de guardar/editar/anotar; verificador bloquea marcadores prohibidos
Electron	`contextIsolation:true`, `nodeIntegration:false`, `sandbox:true`
IPC	preload expone una lista cerrada; lectura valida extensión `.pdf`
Navegación	nuevas ventanas denegadas desde Electron
Android	sin `INTERNET` ni permisos generales de almacenamiento
Android intents	acceso por URI concedida y FileProvider no exportado
Android memoria	apertura externa limitada a 128 MiB
Web	mixed content desactivado en Capacitor
CSP	política local explícita compatible con PDF.js; `object-src` y `frame-src` bloqueados
Búsqueda	contenido y consulta se escapan antes de `innerHTML`
Persistencia	borrado por entrada o completo mediante UI
CI	auditoría de permisos APK, dependencias de producción y advisories del runtime distribuido
Supply chain	versiones fijadas, lockfile congelado, Dependabot agrupado por ecosistema
Release	`zipalign`, `apksigner`, verificación y SHA-256

Datos personales y privacidad

El contenido, nombre y progreso del PDF pueden ser sensibles. Permanecen en memoria/IndexedDB local salvo que la persona comparta voluntariamente el nombre/progreso. No existe cifrado de aplicación; se depende del aislamiento del perfil y del sistema operativo. GitHub Pages puede registrar accesos al sitio, pero el código no sube archivos ni integra telemetría.

Validación y sanitización

La selección Web revisa extensión o MIME; Electron exige extensión; Android confía en intent/MIME y parser PDF.js. Esto no valida la firma mágica antes de cargar. PDF.js trata el formato y devuelve error. Los snippets usan `escapeHtml`; el resto de nombres se escribe con `textContent`.

Riesgos

- PDF malicioso contra una vulnerabilidad de PDF.js/WebView/Electron: mantener dependencias actualizadas y revisar advisories.
- DoS de memoria: Web/Windows no tienen límite explícito y Android Base64 multiplica copias.
- IPC `openPath`: cualquier código que comprometiera el renderer podría solicitar lectura de una ruta `.pdf`; el sandbox/preload reducen, no eliminan, este impacto.
- Persistencia local sin cifrado ni expiración temporal.
- `allowBackup=true` puede permitir que Android incluya datos privados según política/dispositivo; requiere decisión explícita.
- Windows sin Authenticode: SmartScreen y menor garantía de procedencia.
- Las GitHub Actions se referencian por tags mayores (`@vN`) mutables, no por SHA inmutable; una cuenta upstream comprometida podría alterar el código ejecutado por CI.

CORS, CSRF e inyección

No hay endpoints ni cookies de sesión, por lo que CSRF no aplica. CORS no se configura porque no se consume una API del producto. Sí aplican XSS/HTML injection: el único `innerHTML` con datos del PDF pasa por escape; `defaultText.innerHTML` usa texto constante. No se encontraron SQL, `eval` de contenido ni comandos shell formados desde datos del PDF. Android ejecuta un JavaScript constante al anunciar intent nuevo.

Actualizaciones de dependencias por CVE

Los cambios de versión motivados por vulnerabilidades publicadas se registran en Actualizaciones de seguridad por CVE, con identificador CVE y GHSA, severidad, versión que corrige y exposición real del producto.

Fecha	Dependencia	Cambio	Resultado
2026-11-01	`electron`	44.0.0 → 44.10.1	ocho advisories corregidos, uno de severidad alta

La cobertura de la auditoría automática tiene dos capas, porque una sola no basta. `pnpm audit --prod --audit-level high` recorre el árbol de producción: `@capacitor/core`, `@capacitor/share` y `pdfjs-dist`. `pnpm verify:runtime` consulta OSV.dev con la versión exacta de las dependencias que se distribuyen dentro de un artefacto instalable, lo que añade Electron: está declarado en `devDependencies` porque solo se necesita para construir y ejecutar, pero su binario viaja dentro del instalador de Windows, de modo que sus advisories no aparecían en la auditoría de producción y convivieron con un CI en verde.

El árbol completo de desarrollo acumula hallazgos concentrados en el utillaje de build (`electron-builder` y sus transitivas; `rimraf`, `glob` y `tar` bajo `@capacitor/cli` y `@capacitor/assets`). No se usa como gate porque su corrección depende de releases de terceros ajenos al repositorio y porque ese código no se distribuye; se revisa al

preparar cada release.

Secretos y respuesta

La búsqueda estática no identifica secretos reales versionados; solo nombres de GitHub Secrets. Las vulnerabilidades deben informarse mediante Security Advisories sin adjuntar PDF privados. No hay logging/auditoría de runtime; los workflows conservan logs de build.

Recomendaciones prioritarias

1. decidir y probar `android:allowBackup` respecto del historial;
2. imponer límites configurables de tamaño en todas las plataformas;
3. incorporar fuzz/regresión adicional con corpus no sensible y ejecutar los E2E locales dentro de CI;
4. firmar Windows con Authenticode cuando exista infraestructura de certificados;
5. fijar Actions a SHA completos y mantener comentarios con su versión legible.